invisible to the user. You can do what you want with that data. You can use it to perform calculations or update parts of a page. Or you can provide the data as a file download. The data itself can be in many forms. It can be HTML or structured as XML or, more commonly, as JSON. In this chapter, you will look at how to use page handler methods to generate HTML and JSON data from AJAX requests and learn about the limitations of this approach. You will also look at how to post data to page handlers successfully.

If you want to work with JSON extensively, the recommended path is building Web API controllers into your application, which provide the foundation for RESTful HTTP services. I don't cover API controllers in this book, but we will explore a newly simplified request-handling feature introduced in .NET 6 (based on the minimal API) that offers a similar bang for your buck with considerably less ceremony. We will also look at another new feature in .NET 6 that enables you to isolate CSS styles to the scope of a single page without increasing the number of HTTP requests. Before we get into some code examples, we will discuss how you might approach your client-side development.

## *11.1 Choosing your client-side technology*

The jQuery library is, without a doubt, the most used JavaScript library of all time. Introduced in 2006, it quickly became the de facto way to manipulate the DOM, handle events, manage CSS transitions, and perform asynchronous HTTP requests (AJAX). When jQuery was launched, implementations of these APIs differed widely across browsers. jQuery acted as an adapter, providing a set of APIs that worked the same way across all supported browsers.